

Goal Archive
Object Design Document
Versione 1.1



Goal Archive

Data: 24/01/2026

Progetto: Goal Archive	Versione: 1.1
Documento: Object Design Document	Data: 24/01/2026

Coordinatore del progetto:

Nome	Matricola
Sciacca Claudio	0512119806

Partecipanti:

Nome	Matricola
Mediatore Luca	0512119002
La Pastina Domenico	0512119884

Scritto da:	Mediatore Luca, Sciacca Claudio, La Pastina Domenico
--------------------	--

Revision History

Data	Versione	Descrizione	Autore
20/12/2025	1.0	Scrittura Object Design Document	Mediatore Luca, Sciacca Claudio, La Pastina Domenico
24/01/2026	1.1	Revisione generale	Mediatore Luca, Sciacca Claudio, La Pastina Domenico

Indice

1.	INTRODUZIONE	4
1.1.	Object Design Trade-offs	4
1.2.	Linee Guida per la documentazione delle interfacce	5
1.3.	Definizioni, acronimi e abbreviazioni.....	5
1.4	Riferimenti	5
2.	PACKAGES	6
3.	CLASS INTERFACES.....	11
3.1	ManagerUtente	11
3.2	ManagerRosa.....	12
3.3	ManagerPalmares	12
3.4	ManagerCommenti.....	12
3.5	ControlLogin	13
3.6	ControlVisualizzazioneRosa	13
3.7	ControlAggiungiTrofeo.....	13
4.	GLOSSARIO.....	14

1. INTRODUZIONE

Dopo la realizzazione dei documenti RAD e SDD abbiamo descritto, in linea di massima, quello che sarà il nostro sistema e quindi i nostri obiettivi, tralasciando gli aspetti dell'implementazione. L'ODD ha lo scopo di produrre un modello che integri in modo preciso tutte le diverse funzionalità delle fasi precedenti. Questo documento andrà a descrivere in particolare i trade-off, le linee guida sulle interfacce e le convenzioni di codifica.

1.1. Object Design Trade-offs

Trade-off	Scelta progettuale
Affidabilità vs Flessibilità	Scegliamo l'affidabilità in quanto un dato incoerente è inaccettabile come definito nel criterio di robustezza. Il compromesso è che si riduce la flessibilità nell'inserimento di dati parziali o non strutturati, preferendo rifiutare un inserimento incompleto piuttosto che avere un archivio corrotto.
Accessibilità vs Controllo degli Accessi	Per soddisfare il criterio di disponibilità e usabilità scegliamo l'accessibilità in quanto, i dati storici sono visibili anche agli utenti non registrati. Si rinuncia al tracciamento dettagliato di chi consulta i dati privilegiando la facilità d'uso e la diffusione delle informazioni per l'utente non registrato.
Spazio vs Velocità	Scegliamo l'ottimizzazione della velocità; per garantire un basso tempo di risposta, il sistema memorizza direttamente le statistiche aggregate anziché calcolarle in tempo reale ogni volta interrogando le tabelle. Questa scelta occupa maggiore spazio su disco, ma elimina il carico computazionale del calcolo in tempo reale, rendendo la visualizzazione del dato interessato istantanea.
Manutenibilità vs Rapidità di Sviluppo Iniziale	Scegliamo la manutenibilità utilizzando un'architettura modulare (MVC). In linea con i criteri di estensibilità e modificabilità, il sistema separa la logica, i dati e la presentazione. Ciò però comporta ad una complessità strutturale maggiore e tempi di sviluppo iniziali più lunghi rispetto a una soluzione non modulare, ma garantisce una manutenzione più semplice nel tempo.

1.2. Linee Guida per la documentazione delle interfacce

Gli sviluppatori seguiranno alcune linee guida per la scrittura del codice:

- Naming convention: è buona norma utilizzare nomi descrittivi, di uso comune, di lunghezza medio-corta usando solo caratteri consentiti
- Variabili: i nomi delle variabili cominciano con una lettera minuscola e le parole seguenti iniziano con la lettera maiuscola.
- Metodi: i nomi dei metodi iniziano con la lettera minuscola e le successive parole con la maiuscola. Il nome del metodo consiste in un verbo che descrive l'azione che esegue.
- Classi: i nomi delle classi iniziano con lettera maiuscola come anche le successive parole. I nomi di queste ultime devono fornire informazioni sul loro scopo.

1.3. Definizioni, acronimi e abbreviazioni

RAD : Requirements Analysis Document

SDD : System Design Document

ODD : Object Design Document

1.4 Riferimenti

Object Oriented Software Engineering - Using UML, Pattern and Java.

RAD del progetto Goal Archive.

SDD del progetto Goal Archive.

2. PACKAGES

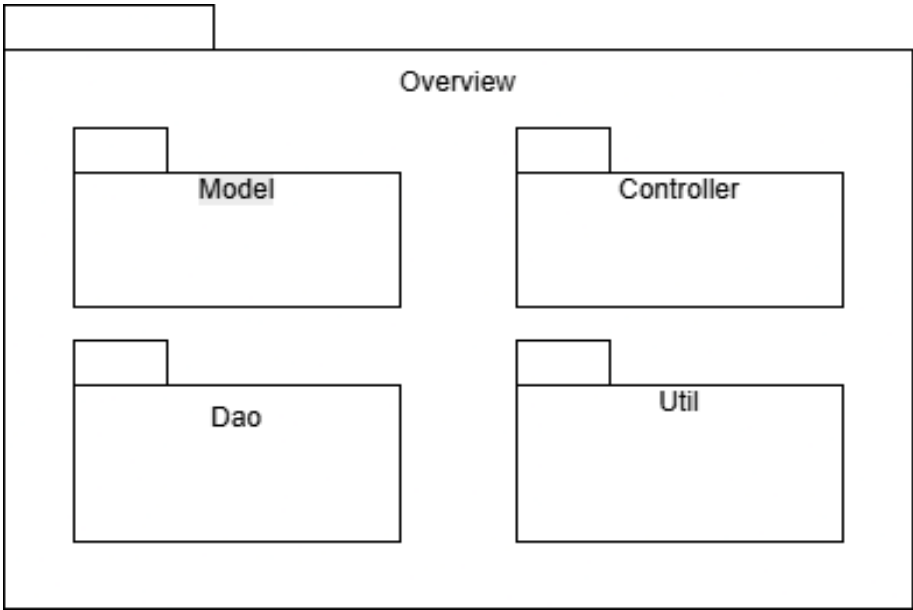
Il nostro sistema presenta una suddivisione basata su tre livelli (three-tier):

- Interface layer
- Application Logic layer
- Storage layer.

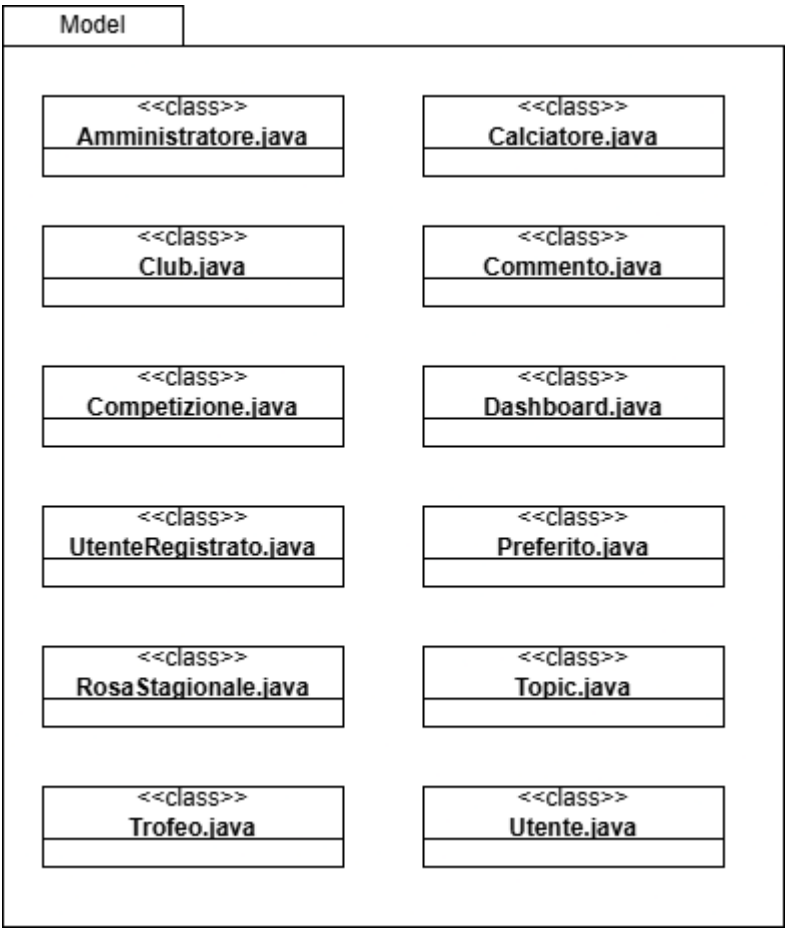
Il package GoalArchive contiene sottopackage che a loro volta inglobano classi atte alla gestione delle richieste utente. Le classi contenute nel package svolgono il ruolo di gestore logico del sistema.

Interface Layer	Rappresenta l'interfaccia del sistema, ed offre la possibilità all'utente di interagire con quest'ultimo, offrendo sia la possibilità di inviare, in input, che di visualizzare, in output, dati.
Application Logic Layer	Ha il compito di elaborare i dati da inviare al client, e spesso grazie a delle richieste fatte al database, tramite lo Storage Layer, accede ai dati persistenti. Si occupa di varie gestioni quali: • Gestione Club • Gestione Palmares • Gestione Campionati • Gestione Nazioni
Storage Layer	Ha il compito di memorizzare i dati sensibili del sistema, utilizzando un DBMS, inoltre riceve le varie richieste dall' Application Logic Layer inoltrandole al DBMS e restituendo i dati richiesti.

2.1 Package Overview

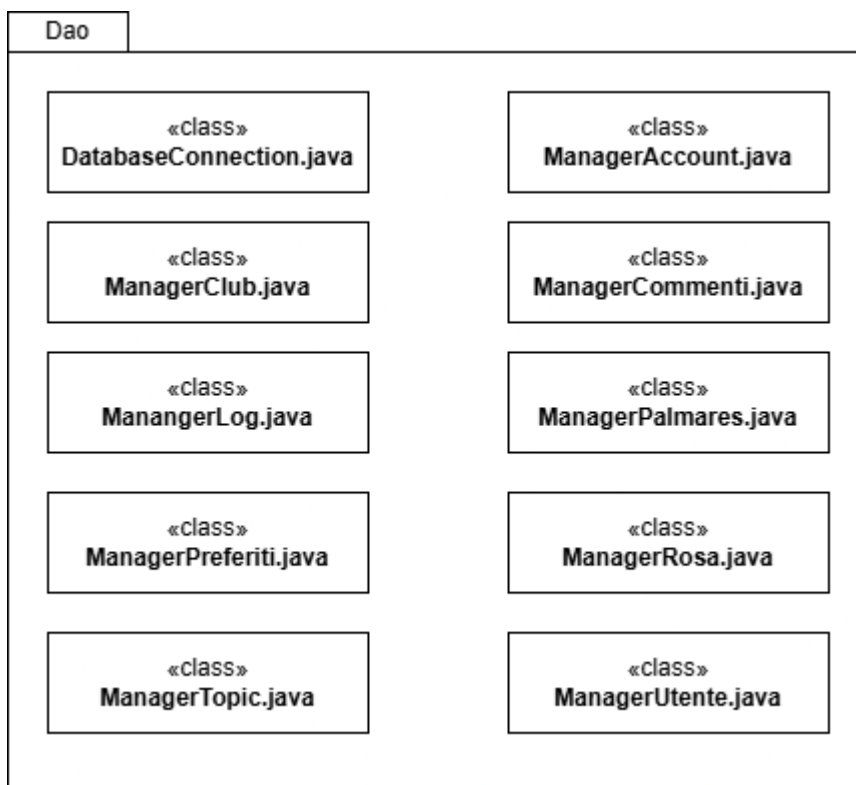


2.2 Package Model



Classe:	Descrizione:
Gestore.java	Questa classe rappresenta il Gestore
Calciatore.java	Questa classe rappresenta il Calciatore
Club.java	Questa classe rappresenta il Club
Commento.java	Questa classe rappresenta un commento
Competizione.java	Questa classe rappresenta una competizione
Dashboard.java	Questa classe rappresenta la dashboard
Preferito.java	Questa classe rappresenta il club preferito
RosaStagionale.java	Questa classe rappresenta una rosa stagionale
Topic.java	Questa classe rappresenta un topic di discussione
Utente.java	Questa classe rappresenta un utente
UtenteRegistrato.java	Questa classe rappresenta un utente registrato

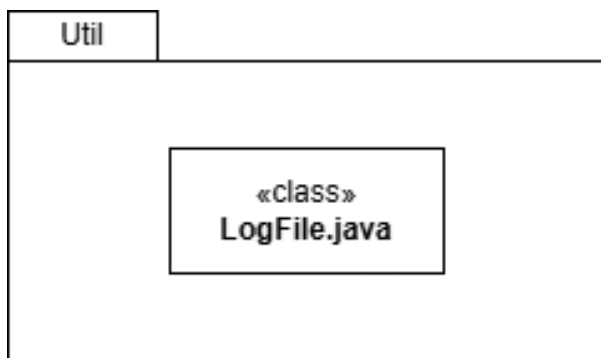
2.3 Package Dao



Classe:	Descrizione:
DatabaseConnection.java	Questa classe rappresenta la connessione al Database
ManagerAccount.java	Questa classe gestisce gli account nel Database
ManagerClub.java	Questa classe gestisce i club nel Database
ManagerCommenti.java	Questa classe gestisce i commenti nel Database
ManagerPalmares.java	Questa classe gestisce i palmares nel Database

ManagerPreferiti.java	Questa classe gestisce i preferiti nel Database
ManagerRosa.java	Questa classe gestisce le rose nel Database
ManagerTopic.java	Questa classe gestisce i topic nel Database
ManagerUtente.java	Questa classe gestisce gli utenti nel Database

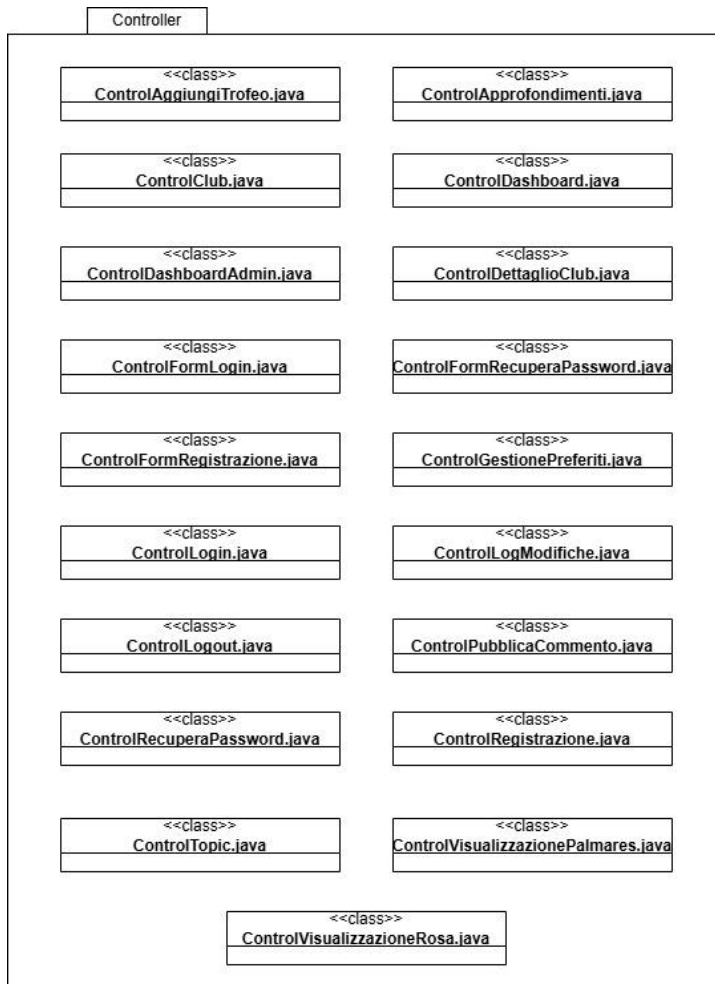
2.4 Package Util



Classe:	Descrizione:
LogFile.java	Questa classe rappresenta il log delle modifiche

2.5 Package Controller

	Ingegneria del Software	Pagina 9 di 14
--	-------------------------	----------------



Classe:	Descrizione:
ControlAggiungiTrofeo.java	Questa classe rappresenta la servlet per aggiungere un trofeo
ControlApprofondimenti.java	Questa classe rappresenta la servlet per la sezione approfondimenti
ControlClub.java	Questa classe rappresenta la servlet per i club
ControlDashboard.java	Questa classe rappresenta la servlet per la dashboard
ControlDashboardAdmin.java	Questa classe rappresenta la servlet per la dashboard del gestore
ControlDettaglioClub.java	Questa classe rappresenta la servlet per visualizzare in dettaglio un club
ControlFormLogin.java	Questa classe rappresenta la servlet per la form del login
ControlFormRecuperaPassword.java	Questa classe rappresenta la servlet per la validazione dati per recuperare la password
ControlFormRegistrazione.java	Questa classe rappresenta la servlet per la form di registrazione
ControlGestionePreferiti.java	Questa classe rappresenta la servlet per la gestione dei preferiti di un utente

ControlLogin.java	Questa classe rappresenta la servlet per il login
ControlLogModifiche.java	Questa classe rappresenta la servlet per il log delle modifiche
ControlLogout.java	Questa classe rappresenta la servlet per il logout
ControlPubblicaCommento.java	Questa classe rappresenta la servlet per la pubblicazione di un commento
ControlRecuperaPassword.java	Questa classe rappresenta la servlet per il recupero della password
ControlRegistrazione.java	Questa classe rappresenta la servlet per la registrazione
ControlTopic.java	Questa classe rappresenta la servlet per i topic
ControlVisualizzazionePalmares.java	Questa classe rappresenta la servlet per la visualizzazione di un palmares
ControlVisualizzazioneRosa.java	Questa classe rappresenta la servlet per la visualizzazione di una rosa

3. CLASS INTERFACES

3.1 *ManagerUtente*

doSave(UtenteRegistrato utente)

Pre-condition:

- utente != null
- utente.email != null
- utente.password != null
- non esiste altro utente con la stessa email nel database.

Post-condition:

- l'utente è inserito nella tabella UtenteRegistrato.
- la password è salvata in forma cifrata.

doRetrieveByEmail(String email)

Pre-condition:

- email != null.

Post-condition:

- ritorna l'oggetto UtenteRegistrato associato a email, se presente.

doDelete(String email)

Pre-condition:

- esiste un utente con l'email specificata.

Post-condition:

- l'utente non è più presente nella tabella UtenteRegistrato.

3.2 ManagerRosa

doRetrieveByClubAndStagione(int idClub, String stagione)

Pre-condition:

- idClub valido e presente in Club.
- stagione non nulla.

Post-condition:

- ritorna una lista di oggetti RosaStagionale con i relativi Calciatore per quel club e stagione.

3.3 ManagerPalmares

doRetrieveByClub(int idClub)

Pre-condition:

- idClub valido.

Post-condition:

- ritorna la lista di Trofeo associata al club, raggruppata per competizione.

doSaveTrofeo(Trofeo trofeo, String motivo)

Pre-condition:

- trofeo non nullo e referenze a Club e Competizione valide.
- non esiste già un trofeo identico (stessa competizione e stagione) per il club.
- motivo non vuoto (obbligatorio per il log).

Post-condition:

- trofeo inserito nella tabella Trofeo.
- nuova entry nel LogModifica con motivo e utente gestore.

3.4 ManagerCommenti

doSave(Commento commento)

Pre-condition:

- commento non nullo, utente registrato autenticato.
- testo del commento non vuoto.

Post-condition:

- commento inserito nella tabella Commento con data di pubblicazione.

doRetrieveByTopic(int idTopic)**Pre-condition:**

- idTopic valido.

Post-condition:

- ritorna l'elenco dei commenti associati al topic, ordinati per data crescente.

3.5 ControlLogin

processRequest(request, response)**Pre-condition:**

- la request contiene parametri email/username e password.

Post-condition:

- se le credenziali sono valide, viene creata una sessione con attributi di ruolo (utente o gestore) e l'utente viene reindirizzato alla dashboard corrispondente; in caso contrario viene mostrato un messaggio di errore.

3.6 ControlVisualizzazioneRosa

processRequest(request, response)**Pre-condition:**

- la request contiene identificativo del club e stagione selezionata, ottenuti dalla pagina club/archivioRose.jsp.

Post-condition:

- la JSP archivioRose.jsp riceve la lista dei giocatori organizzata per ruolo e le statistiche principali da mostrare all'utente.

3.7 ControlAggiungiTrofeo

processRequest(request, response)**Pre-condition:**

- utente autenticato come gestore.

- form con campi competizione, stagione, note e motivo della modifica compilato correttamente.

Post-condition:

- nuovo trofeo registrato per il club selezionato e log aggiornato; la pagina palmares.jsp mostra il trofeo appena inserito.

4. GLOSSARIO

- **RAD:** Documento di Analisi dei Requisiti.
- **DBMS:** Sistema di gestione di basi di dati.
- **SDD:** Documento di System Design.
- **ODD:** Documento di Object Design.
- **Database:** Insieme organizzato di dati persistenti.
- **Query:** Termine utilizzato per indicare l'interrogazione da parte di un utente di un database.
- **Trade-off:** Compromesso progettuale tra due requisiti contrastanti effettuato per ottimizzare il sistema secondo i criteri di successo stabiliti.
- **Pre-condition / Post-condition:** Vincoli formali che devono essere soddisfatti rispettivamente prima e dopo l'esecuzione di un metodo nelle interfacce dei Manager per garantirne la robustezza.
- **Manager (DAO):** Classi specializzate che implementano i metodi di accesso ai dati e gestiscono la persistenza degli oggetti del modello.
- **Control (Controller):** Classi che mediano tra la vista (JSP) e la logica di business, elaborando le request HTTP e indirizzando il flusso verso la response corretta.